Evidence-Aware Memory for Coding Agents on Real Repository History: A Leakage-Controlled Evaluation

Mihit
IILM University, Greater Noida, India

Abstract

Coding agents that keep memory about a repository must cope with knowledge that goes stale, is contradicted, or was never reliable. We study EmemoVeri, a memory mechanism that attaches typed, timestamped evidence to repository claims, evolves a claim state (provisional, corroborated, refined, contested, superseded), maintains a trust value, and ranks claims with an evidence-, staleness- and scope-aware score. Earlier evaluations of this design relied on simulated scenarios whose generator shares structure with the scoring function, so they cannot support real-world claims. We replace them with a benchmark that replays the git history of three open-source repositories (fastapi, yt-dlp, browser-use) on one chronological clock and asks a memory system to rank the files that a *future* commit will modify, given only its subject line and memory built from strictly earlier history. Reference files come from git, relations between commits and claims come from structural events (add, modify, delete, rename, revert) rather than keywords, and the harness includes a leak canary, a shuffled-query control, and a strict dev/test split in time. On 768 held-out tasks, a dev-tuned EmemoVeri configuration improves Hit@5 over a snapshot-aware similarity baseline by 0.111 (95% CI [0.082, 0.142]), and state-aware retrieval removes the dead-path suggestions (0.179 of top-5 slots) that unfiltered retrieval produces. The picture is less favourable for parts of the original design: with the blueprint weights the gain over snapshot retrieval is small and not significant after correction (0.043, Holm $p = 0.112$), the trust term is saturated and uninformative, the tuner sets its weight to about zero, and the abstention rule is no better than a plain similarity gap. We release the code, tests, and all result files, and document eight defects that we found and fixed in the original implementation.

1 Introduction

A coding agent that works on the same repository over weeks accumulates knowledge: which module owns a behaviour, which API replaced a deprecated one, which test guards an invariant. Such knowledge decays. Files are renamed or deleted, changes are reverted, and conclusions drawn from one commit are overturned by the next. A memory that simply stores and retrieves text treats an outdated statement and a current one alike. EmemoVeri is a design that tries to address this by tracking *evidence* for each remembered claim and letting the claim's *state* change as evidence arrives.

The design is easy to describe and hard to evaluate honestly. Real tasks require a model in the loop, long runs, and labels that do not depend on the memory under test. Synthetic scenarios are cheap, but when the scenario generator decides which claim is correct by giving it higher trust, fresher timestamps and test evidence, it rewards exactly the features the scorer uses. We found that this was true of the original EmemoVeri experiments, and we do not use them as evidence here.

This paper makes four contributions.

1. **A leakage-controlled replay benchmark on real history.** Memory is built commit by commit on a single chronological clock; every task is scored before its own commit is ingested; relations come from structural git events; reference files come from git. Section 4 describes the construction,

a leak canary that verifies the harness detects leakage, and a shuffled-query control that quantifies how much of a score is independent of the query.

2. **A comparison against baselines that matter.** Besides plain and naive retrieval we include *snapshot RAG*, a similarity baseline that discards files absent from the current checkout. This is what an agent with a working tree obtains at no cost, and it absorbs most of the benefit of knowing that a file is gone.

3. **A component analysis with null results reported.** We ablate the trust, evidence, staleness, state and cross-repository components, evaluate selective answering, and report where the original design does not help.

4. **An audit of the original implementation.** Eight defects, including a leakage guard that leaked, a supersession state that was not terminal, and retrieval that ignored claim state, are documented, fixed, and covered by regression tests.

We do not claim that EmemoVeri improves end-to-end agent success; no language model is in the loop. We claim only what the benchmark measures: how well a memory mechanism localises future changes from earlier history, and what each component contributes to that.

2 Related work

Memory for language-model agents. Retrieval-augmented generation [1] grounds generation in retrieved text. Agent-memory systems add write and update policies: MemGPT manages a tiered context [2], Generative Agents retrieve by recency, importance and relevance [3], Reflexion stores verbal feedback across trials [4], and recent systems such as Mem0 [5] and A-MEM [6] organise and update memories dynamically. LongMemEval measures long-term memory abilities of chat assistants, including knowledge updates [7]. Our setting differs in that the memory is about a code repository, updates come from version-control events, and the ground truth can be read from the history itself.

Repository-level code assistance. SWE-bench evaluates agents on real issues [8], SWE-agent studies agent–computer interfaces [9], and RepoCoder retrieves repository context iteratively for completion [10]. These evaluate task success with a model in the loop. We evaluate a memory component in isolation, at lower cost and lower realism.

Mining software repositories. Using change history to guide edits has a long record: co-change mining suggests related files [11], cached history and code-change complexity predict where faults occur [12, 13], and information-retrieval bug localisation ranks files against a report [14]. A recurring lesson, which our results echo, is that simple activity priors such as recency and change frequency are strong. Common pitfalls of repository mining are discussed by Kalliamvakou et al. [15]. We read history with plain `git` in the spirit of tools such as PyDriller [16].

Selective prediction and calibration. Answering only when confident is studied as selective classification [17]; calibration is commonly summarised by expected calibration error [18]. We use both to assess the trust value and the abstention rule.

3 The EmemoVeri mechanism

Claims and evidence. A claim has a statement, a scope, a state, and a trust value in $[0, 1]$. Evidence items have a type (commit, test, issue, documentation, agent observation, external), a relation to the claim (corroborates, refines, contradicts, supersedes), a timestamp, and a strength. Each type has a reliability prior, and each relation has a sign.

State. A claim starts PROVISIONAL and moves to CORROBORATED, REFINED or CONTESTED as evidence accumulates. SUPERSEDED is terminal: later evidence is recorded for provenance but cannot revive the claim. (In the original implementation this was not the case; see Section 8.)

Trust. The default update is sequential and clamped, $t \leftarrow \mathrm{clip}(t + \eta\, w_e, 0, 1)$ with learning rate $\eta = 0.35$. We also implement a pooled clamped update and an order-invariant, non-saturating update, $t = \sigma(\mathrm{logit}(t_0) + \eta \sum_e w_e)$.

Retrieval score. For a query with scope q at reference time τ,

$$s = 0.45\,\mathrm{sim} + 0.25\,\mathrm{trust} + 0.20\,\mathrm{evid} + 0.10\,\mathrm{scope} - 0.10\,\mathrm{stale}, \tag{1}$$

where $\mathrm{evid} = E/(E+5)$ for total evidence strength E, $\mathrm{stale} = 1 - 2^{-\Delta/H}$ with Δ the time since the last update and $H = 90$ days, and scope is 1 for a same-scope claim and 0 otherwise. SUPERSEDED claims are excluded. A local-first policy admits a cross-scope claim only if its score exceeds the best local score by a margin.

Abstention. A conflict resolver answers with the top claim when the score gap to the runner-up is at least 0.20, and abstains otherwise.

4 Benchmark

4.1 Task

At time t the system receives the cleaned subject line of a *future* commit C and must rank source files. Memory contains only information from commits strictly before C. The reference set is the set of source files that C modified. No classifier is involved in producing it. Because the query is the subject of a commit that has not yet happened, it stands in for a task description; it is more specific than a typical issue and we return to this in Section 7.

4.2 Memory from structural events

There is one claim per live `.py` file under the repository's source root. Relations are read from git's own status letters rather than inferred from wording: an addition corroborates, a modification refines, a deletion or a rename away supersedes, and a commit carrying git's "This reverts commit" trailer contradicts the claims of the files it touches. A rename carries the file's evidence and text to the new path and supersedes the old claim; a path that reappears after deletion becomes a new claim. Evidence is of type TEST when the commit also edits a test file and COMMIT otherwise, with strength 1.0, or 0.3 for bulk commits that touch more than three source files. Claim text is the file path plus the five most recent cleaned subjects of commits touching it. Similarity is cosine over hashed unigram and bigram TF-IDF vectors in which IDF is recomputed from pre-cutoff claims for every task, so no statistic is fitted on later data.

Commits are replayed in topological order. Real histories contain non-monotone timestamps (Table 1), so the clock is the running maximum of committer timestamps. Table 2 shows how often each structural event occurs: deletions and renames are common in yt-dlp, but reverts that touch source files are rare, so the contradiction path is exercised only lightly.

4.3 Integrity controls

Memory is updated only after a task has been scored, and a runtime assertion rejects any memory state containing evidence newer than the task. As a *leak canary*, each task is re-scored after its own commit has been ingested; a harness that can detect leakage must show a large jump, and ours does (Hit@1

Repository	Commits	Source-touching	Eligible	Selected	Cold-file excl.	Clock anomalies
fastapi	7,712	853	296	296	0	12
yt-dlp	22,449	5,052	1,885	400	0	2,289
browser-use	7,003	4,057	1,773	400	5	4,294

Table 1: Mined repositories and task selection. "Eligible" tasks are non-bot, non-revert, modify one to four source files that already have a claim, and occur after a warm-up of 35% of the repository's source-touching commits; up to 400 evenly spaced tasks per repository are selected. Clock anomalies are commits whose recorded timestamp precedes the running clock. History of yt-dlp that predates its `yt_dlp/` package directory is outside the source root and therefore not tracked.

Repository	Adds	Modifications	Deletions	Renames	Reverts (touching source)
fastapi	59	1,476	7	1	4 (3)
yt-dlp	528	12,324	327	853	51 (5)
browser-use	258	6,651	103	47	20 (15)

Table 2: Structural events on source files in the full histories.

0.999 for snapshot RAG and 0.999 for EmemoVeri). As a *shuffled-query control*, each task is scored with another task's query. Weights are tuned only on the earliest 30% of tasks on the global clock (328 dev tasks, 768 test tasks); a 420-point grid over similarity, trust and evidence weights, staleness weight and half-life, and trust update mode is searched by dev MRR. browser-use appears only in the test period, so it serves as a repository unseen during tuning.

4.4 Conditions and metrics

Query-independent priors: random live file, most recently updated, most evidence. **Plain RAG:** similarity only, over all repositories, dead files included. **Naive memory:** similarity only, local repository, dead files included. **Snapshot RAG:** similarity only, local repository, dead files removed. **EmemoVeri:** the score above with blueprint weights. **EmemoVeri tuned:** dev-selected weights. We report Hit@k (a reference file among the top k distinct files), MRR, the fraction of top-5 slots that are dead paths, and the fraction that come from another repository. Differences are paired at task level with 10,000-sample bootstrap intervals; binary outcomes use an exact McNemar test with Holm correction over the 16 binary comparisons.

5 Results

5.1 Comparison with baselines

Table 3 and Figure 1 give test-set results. Unfiltered retrieval suffers from two hazards that are easy to measure here. Plain RAG places a file from another repository in 0.267 of its top-5 slots, and both plain RAG and naive memory place a deleted or renamed-away path in about 0.179 of them; at query time about 20% of all claims are dead, and 54% of test tasks have at least one dead path in the plain-RAG top five. Removing dead files alone (snapshot RAG) raises Hit@5 by 0.090 over plain RAG (Figure 5).

With the blueprint weights, EmemoVeri improves Hit@5 over plain RAG by 0.133 (95% CI [0.090, 0.176]) but over snapshot RAG by only 0.043 (95% CI [0.000, 0.086], Holm $p = 0.112$), which is not significant after correction (Table 4). Most of what state-aware memory offers over naive retrieval is therefore available to any agent that can see its working tree. The dev-tuned configuration, in contrast, improves Hit@5 over snapshot RAG by 0.111 (95% CI [0.082, 0.142]); on 117 tasks only the tuned

Condition	Hit@1	Hit@3	Hit@5	MRR	Dead@5	Other-repo@5
Random (local)	0.016	0.047	0.081	0.051	0.000	0.000
Recency	0.189	0.375	0.451	0.307	0.000	0.000
Popularity	0.083	0.371	0.451	0.244	0.000	0.000
Plain RAG (all repos)	0.333	0.495	0.551	0.433	0.179	0.267
Naive memory (local)	0.357	0.531	0.612	0.470	0.181	0.000
Snapshot RAG	0.378	0.564	0.641	0.497	0.000	0.000
EmemoVeri (default weights)	0.406	0.607	0.684	0.532	0.000	0.000
EmemoVeri (dev-tuned)	0.454	0.661	0.751	0.585	0.000	0.000

Table 3: Test results ($n = 768$). Dead@5 and Other-repo@5 are the fractions of top-5 slots that are dead paths or files from another repository.

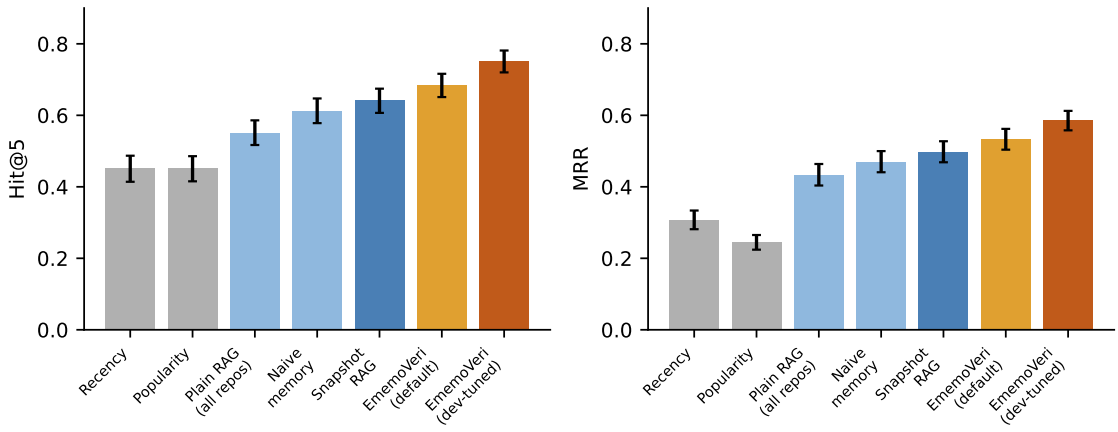


Figure 1: Hit@5 and MRR on the test split with 95% bootstrap intervals.

system hits and on 32 only snapshot RAG hits. The selected weights are 0.8 similarity, 0.0 trust, 0.1 evidence, 0.05 staleness (half-life 365 days): a mostly lexical ranker with a light activity prior.

5.2 Repositories differ

Table 5 and Figure 2 break results down by repository. The default weights help on fastapi and browser-use but hurt on yt-dlp, where Hit@5 falls from 0.801 (snapshot RAG) to 0.689. A likely reason, which we did not test directly, is that yt-dlp has one extractor file per website, so path tokens match queries well and a query-independent prior mostly adds noise. The tuned configuration recovers this (0.893), and its gain holds on browser-use, which tuning never saw (0.605 for snapshot RAG against 0.698 tuned). On fastapi, popularity alone reaches 0.640, comparable to the default EmemoVeri (0.645), because a few hot files account for most edits. Any aggregate number should be read with this heterogeneity in mind.

5.3 What each component contributes

Table 6 and Figure 3 ablate the default system. The state-aware filter does what it is designed to do for staleness: without it, 0.015 of top-5 slots are dead paths instead of none, but Hit@5 barely changes because dead files are almost never reference files. Removing the trust term or the staleness term *improves* Hit@5 (+0.021 and +0.035), a longer half-life helps, and a shorter one does not (its interval includes zero). The evidence term's effect is not distinguishable from zero. The three cross-repository policies

A	B	Metric	Δ (A−B) [95% CI]	A-only / B-only	p_{Holm}
EmemoVeri (default weights)	Plain RAG (all repos)	Hit@1	+0.073 [+0.036, +0.109]	132 / 76	0.0008
EmemoVeri (default weights)	Plain RAG (all repos)	Hit@5	+0.133 [+0.090, +0.176]	203 / 101	<0.0001
EmemoVeri (default weights)	Naive memory (local)	Hit@1	+0.049 [+0.013, +0.086]	121 / 83	0.0282
EmemoVeri (default weights)	Naive memory (local)	Hit@5	+0.072 [+0.029, +0.115]	171 / 116	0.0070
EmemoVeri (default weights)	Snapshot RAG	Hit@1	+0.029 [−0.009, +0.065]	113 / 91	0.1413
EmemoVeri (default weights)	Snapshot RAG	Hit@5	+0.043 [+0.000, +0.086]	157 / 124	0.1121
EmemoVeri (dev-tuned)	Snapshot RAG	Hit@1	+0.077 [+0.052, +0.103]	82 / 23	<0.0001
EmemoVeri (dev-tuned)	Snapshot RAG	Hit@5	+0.111 [+0.082, +0.142]	117 / 32	<0.0001
EmemoVeri (dev-tuned)	EmemoVeri (default weights)	Hit@1	+0.048 [+0.020, +0.077]	84 / 47	0.0070
EmemoVeri (dev-tuned)	EmemoVeri (default weights)	Hit@5	+0.068 [+0.038, +0.100]	105 / 53	0.0003
Snapshot RAG	Plain RAG (all repos)	Hit@1	+0.044 [+0.031, +0.060]	34 / 0	<0.0001
Snapshot RAG	Plain RAG (all repos)	Hit@5	+0.090 [+0.070, +0.111]	69 / 0	<0.0001

Table 4: Paired comparisons on the test split. A-only / B-only count tasks where only A or only B has a hit.

Repository (n)	Condition	Hit@1	Hit@5	MRR
browser-use (400)	Popularity	0.147	0.463	0.289
	Plain RAG (all repos)	0.258	0.502	0.364
	Snapshot RAG	0.315	0.605	0.442
	EmemoVeri (default weights)	0.393	0.698	0.523
	EmemoVeri (dev-tuned)	0.403	0.698	0.532
fastapi (172)	Popularity	0.012	0.640	0.275
	Plain RAG (all repos)	0.198	0.384	0.293
	Snapshot RAG	0.244	0.541	0.387
	EmemoVeri (default weights)	0.297	0.645	0.464
	EmemoVeri (dev-tuned)	0.326	0.715	0.493
yt-dlp (196)	Popularity	0.015	0.260	0.125
	Plain RAG (all repos)	0.607	0.796	0.697
	Snapshot RAG	0.622	0.801	0.708
	EmemoVeri (default weights)	0.531	0.689	0.611
	EmemoVeri (dev-tuned)	0.673	0.893	0.773

Table 5: Test results by repository.

are indistinguishable in the top five, because the scope term alone keeps other repositories out; the policy only matters if the scope weight is removed, as in plain RAG. The non-saturating trust update does not rescue the trust signal. Taken together, the gain of the tuned system over the default comes mostly from down-weighting trust and staleness and keeping a small evidence prior.

5.4 Trust and abstention

For the top-ranked claim, mean trust is 1.000 under the default update: trust saturates, and its AUROC for predicting a correct top-1 answer is 0.500, no better than chance. Its expected calibration error (0.594) equals one minus accuracy, as expected of a constant near 1. The score gap used by the conflict resolver is informative (AUROC 0.751 default, 0.783 tuned), but a plain similarity gap on snapshot RAG is at least as good (0.820). At the prescribed threshold of 0.20 the default system answers 0.3% of tasks (100.0% correct); the tuned system answers 5.1% (97.4%); snapshot RAG answers 6.2% (93.8%). Lower thresholds trade coverage for accuracy along a curve that is similar across systems (Figure 4, Table 7). We therefore find no evidence that EmemoVeri's abstention rule adds value over a similarity gap in this setting.

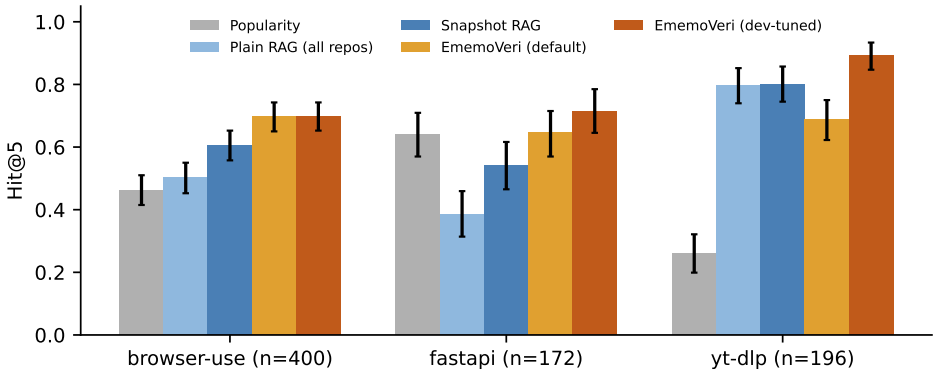


Figure 2: Hit@5 by repository with 95% bootstrap intervals.

Variant of default EmemoVeri	ΔHit@5 [95% CI]	ΔMRR [95% CI]
No trust term	+0.021 [+0.010, +0.033]	+0.007 [+0.001, +0.013]
No evidence term	−0.012 [−0.047, +0.023]	+0.009 [−0.016, +0.033]
No staleness term	+0.035 [+0.018, +0.052]	+0.023 [+0.011, +0.037]
State-blind (keep SUPERSEDED)	−0.001 [−0.004, +0.000]	−0.003 [−0.005, −0.001]
Cross-repo: open	+0.000 [+0.000, +0.000]	−0.000 [−0.001, −0.000]
Cross-repo: local only	+0.000 [+0.000, +0.000]	+0.000 [+0.000, +0.000]
Cross-repo: similarity gate	+0.000 [+0.000, +0.000]	+0.000 [+0.000, +0.000]
Trust: pooled, clamped	+0.004 [+0.000, +0.009]	+0.002 [−0.001, +0.006]
Trust: pooled, logit	−0.007 [−0.016, +0.003]	−0.009 [−0.016, −0.003]
Staleness half-life 30 d	−0.013 [−0.027, +0.001]	−0.017 [−0.027, −0.008]
Staleness half-life 365 d	+0.016 [+0.003, +0.029]	+0.013 [+0.004, +0.022]

Table 6: Ablations relative to default EmemoVeri on the test split. Positive values mean the variant is better than the default.

5.5 Harness validity and robustness

The leak canary reaches Hit@1 above 0.999, so a leaking pipeline would be caught. The shuffled-query control is informative in the other direction: with another task's query, snapshot RAG falls to Hit@5 0.130, but default EmemoVeri keeps 0.461, about the level of recency and popularity alone (0.451 and 0.451). A sizeable part of the default system's score is therefore independent of the query. This is not a flaw in itself, since activity priors are known to be strong predictors in repository mining [12, 13], but it means that credit for the gain should not be assigned to evidence semantics without further study. Table 8 repeats the main comparison with a different warm-up fraction and sample size. The conclusions hold: the tuned system beats snapshot RAG by 0.09 to 0.11 Hit@5 in every setting, the default is never significantly better than snapshot RAG after correction, and the tuner assigns the trust term a weight of at most 0.1.

6 Discussion

What is supported. State-aware filtering removes a real and measurable class of error, suggestions of files that no longer exist, and it removes cross-repository contamination when scope is used. A tuned blend of lexical similarity with a light evidence and recency prior beats a snapshot-aware lexical baseline, consistently across settings and on a repository unseen during tuning.

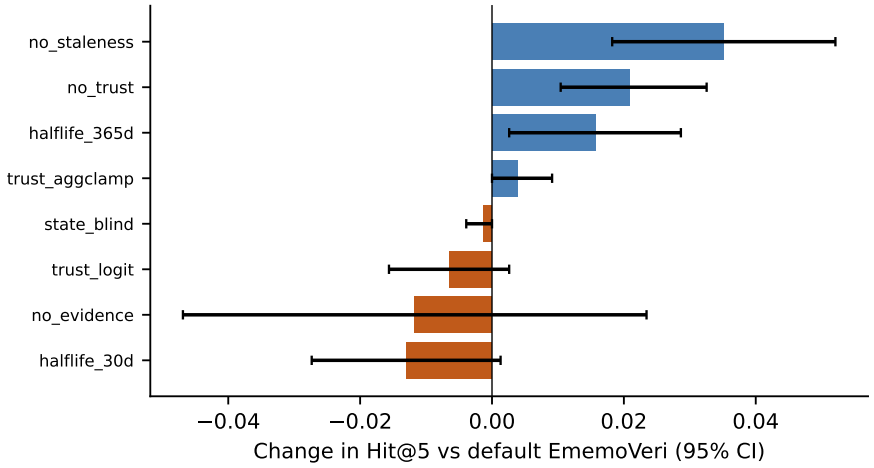


Figure 3: Change in Hit@5 relative to default EmemoVeri (95% bootstrap intervals).

System	Top-1 acc.	AURC (gap) ↓	AUROC (gap)	AUROC (trust)
Snapshot RAG	0.378	0.362	0.820	0.570
EmemoVeri (default weights)	0.406	0.376	0.751	0.500
EmemoVeri (dev-tuned)	0.454	0.307	0.783	0.489

Table 7: Selective answering on the top-1 file. AURC is the mean risk over coverage (lower is better); the random-confidence value equals one minus top-1 accuracy.

What is not. The blueprint weights do not reliably beat snapshot retrieval; the trust value carries no usable signal under the default update, and replacing the update did not fix that; the abstention rule is not better than a similarity gap. Because the benchmark's evidence is almost all positive (modifications), it does not exercise contested states much, and we cannot say whether contradiction handling would matter in settings with more reverts or conflicting sources.

Why the benchmark may understate or overstate. Commit subjects are more informative than issue text, which may favour lexical similarity; real agents also see the working tree, code content and tests, which this benchmark does not use. On the other hand, the recency and popularity priors might matter more when the query is vaguer. We make no claim in either direction.

7 Threats to validity

Construct. File localisation from a commit subject is a proxy for agent tasks; it says nothing about patch quality. *Internal.* Tasks are not independent, since hot files recur and commits cluster by author and time, so bootstrap intervals over tasks are optimistic. Weights were tuned on the same task family as the test set, although in an earlier period and with one repository held out entirely. Tie-breaking favours recently updated claims, which can favour recency-heavy conditions slightly. *External.* Three Python repositories, one of which has an unusual one-file-per-site layout. Results depend on the clone date; we pin repository heads in the release. *Conclusion.* Sixteen binary comparisons were corrected with Holm's method; the bootstrap intervals for paired differences are not corrected.

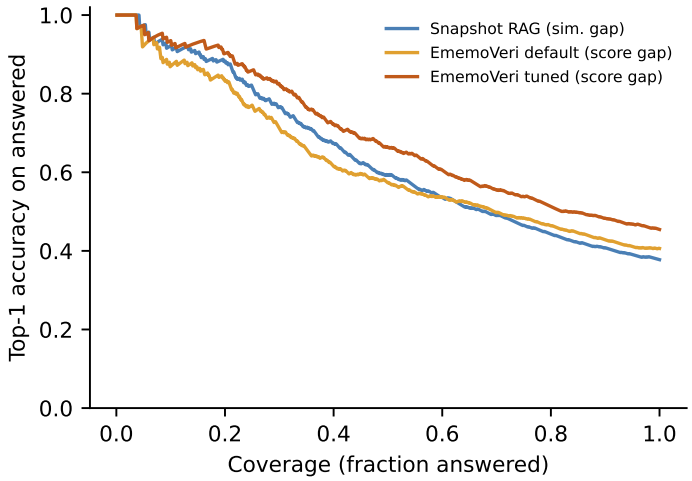


Figure 4: Top-1 accuracy among answered tasks as coverage increases, ordering by score gap.

Setting (test n)	Default vs. Snapshot RAG	Tuned vs. Snapshot RAG	Tuned trust weight
Main: warm-up 0.35, 400/repo (768)	+0.043 [+0.000, +0.086]	+0.111 [+0.082, +0.142]	0.0
Warm-up 0.20 (839)	+0.043 [+0.001, +0.085]	+0.112 [+0.085, +0.141]	0.0
Warm-up 0.50 (712)	+0.007 [−0.035, +0.051]	+0.088 [+0.060, +0.118]	0.0
150 tasks/repo (315)	+0.025 [−0.038, +0.092]	+0.111 [+0.067, +0.159]	0.1

Table 8: Hit@5 differences under alternative settings (95% bootstrap intervals).

8 Audit of the original implementation

While building the benchmark we reproduced and fixed eight defects in the first version of the code. (1) SUPERSEDED was not terminal: a later corroboration moved a superseded claim back to CORROBORATED. (2) Retrieval ignored claim state, so a superseded claim with higher trust could rank first. (3) The temporal leakage guard dropped future evidence but kept the trust and state that had been derived from it; it now replays visible evidence through the trust model and state engine. (4) Staleness defaulted to the wall clock, making every historical claim look fully stale; it now defaults to the knowledge cutoff. (5) The keyword relation classifier mislabels ordinary commits (for example, substring matches on "add"), so real-history relations use structural git events instead. (6) Clamped trust updates saturate after two or three strong items. (7) A single candidate always resolved, regardless of score. (8) The cross-repository split used object equality in one module and identity in another. Each fix has a regression test. Results produced before these fixes, including the original synthetic experiments, should not be reused.

9 Conclusion

On real repository history, a memory that knows which files have disappeared, and that blends lexical similarity with a light evidence and recency prior, localises future changes better than plain retrieval and, once tuned, better than a snapshot-aware baseline. The original design's trust and abstention components did not earn their place in this setting, and the blueprint weights over-weight priors on repositories where the query is highly informative. The natural next steps are a language model in the loop on issue-derived tasks, repositories with more contradictory history, and a trust model that uses evidence beyond counts.

Reproducibility. Code, tests, raw per-task outcomes, all summaries and figures, and pinned repository heads are in the accompanying repository. `scripts/reproduce_real.sh` regenerates every number;

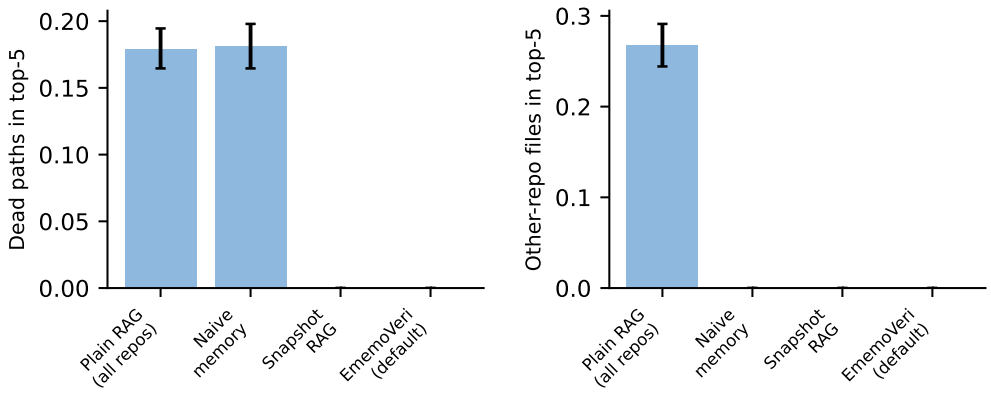


Figure 5: Hazards of unfiltered retrieval: dead paths and other-repository files in the top five.

tables and numeric macros in this paper are produced by `paper/make_tables.py` from `results/real/summary.json`.

Acknowledgments. Parts of the benchmark code and of the manuscript text were prepared with a large language model. The author reviewed the output, ran all experiments, and takes full responsibility for the content.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[2] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[3] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[4] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

[5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[6] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

[7] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[9] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

[10] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[11] Thomas Zimmermann, Peter Weisgerber, Stephan Diehl, and Andreas Zeller. Mining version histories to guide software changes. *IEEE Transactions on Software Engineering*, 2005.

[12] Sunghun Kim, Thomas Zimmermann, E. James Whitehead Jr., and Andreas Zeller. Predicting faults from cached history. In *International Conference on Software Engineering (ICSE)*, 2007.

[13] Ahmed E. Hassan. Predicting faults using the complexity of code changes. In *International Conference on Software Engineering (ICSE)*, 2009.

[14] Jian Zhou, Hongyu Zhang, and David Lo. Where should the bugs be fixed? more accurate information retrieval-based bug localization based on bug reports. In *International Conference on Software Engineering (ICSE)*, 2012.

[15] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Working Conference on Mining Software Repositories (MSR)*, 2014.

[16] Davide Spadini, Maurício Aniche, and Alberto Bacchelli. PyDriller: Python framework for mining software repositories. In *ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2018.

[17] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.

[18] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.